

TP1 : Introduction au neurone linéaire et à la descente de gradient

Machine Learning 2 : Deep Learning

Objectifs du TP

Ce premier TP a pour objectif de comprendre les mécanismes fondamentaux utilisés pour entraîner un modèle de Machine Learning et, par extension, un réseau de neurones.

À la fin de ce TP, vous serez capables de :

- comprendre le fonctionnement d'un neurone artificiel simple ;
- comprendre le rôle des poids et du biais ;
- comprendre la notion de prédiction ;
- utiliser une fonction de perte pour mesurer l'erreur d'un modèle ;
- comprendre le principe de la descente de gradient ;
- comprendre le rôle du *learning rate* ;
- comprendre l'intérêt de la standardisation ;
- séparer les données en ensembles d'entraînement et de validation ;
- utiliser les métriques MSE et MAE ;
- comparer une solution obtenue par descente de gradient avec une solution analytique.

Librairies utilisées : Python 3.9+, NumPy, Pandas, Matplotlib et scikit-learn.

1. Pourquoi commencer par un modèle aussi simple ?

Le Deep Learning repose sur des réseaux pouvant contenir des milliers, des millions, voire des milliards de paramètres.

Avant d'étudier ces architectures complexes, il est important de comprendre **comment un modèle très simple apprend à partir de données**.

Considérons un problème concret.

Nous souhaitons prédire la valeur d'un logement à partir du nombre moyen de pièces.

Nous disposons donc d'une entrée :

```
[  
x = \text{nombre moyen de pièces}  
]
```

et nous voulons prédire une sortie :

```
[
y = \text{valeur du logement}
]
```

Le fonctionnement général est :

```
[
x
\longrightarrow
\boxed{\text{Modèle}}
\longrightarrow
\hat{y}
]
```

où :

- x est l'information donnée au modèle ;
- y est la vraie valeur ;
- $\hat{y}$ est la valeur prédite par le modèle.

Mais au départ, le modèle **ne sait pas comment effectuer correctement cette prédiction**.

Il doit apprendre à partir d'exemples.

Trois questions apparaissent alors.

Question 1 : Comment produire une prédiction ?

Le modèle doit transformer son entrée x pour produire $\hat{y}$.

Nous allons utiliser :

```
[
\hat{y}=wx+b
]
```

Question 2 : Comment savoir si la prédiction est correcte ?

Nous devons comparer :

```
[
\hat{y}
]
```

avec :

[
y
]

et mesurer l'erreur commise.

Nous utiliserons pour cela une **fonction de perte**.

Question 3 : Comment améliorer automatiquement le modèle ?

Si la prédiction est mauvaise, il faut modifier (w) et (b).

Nous utiliserons pour cela la **descente de gradient**.

L'apprentissage peut donc être résumé par :

[
\boxed{
\text{Données}
\rightarrow
\text{Prédiction}
\rightarrow
\text{Erreur}
\rightarrow
\text{Correction}
\rightarrow
\text{Nouvelle prédiction}
}
]

Ce cycle est répété plusieurs fois afin d'améliorer progressivement le modèle.

2. Quel est le lien avec le Deep Learning ?

Dans ce TP, nous allons travailler avec un seul neurone et une seule entrée :

[
x \rightarrow wx+b \rightarrow \hat{y}
]

Un réseau de neurones contient plusieurs neurones organisés en couches :

[
\text{Entrées}
\rightarrow
\text{Couche 1}
\rightarrow

```

\text{Couche 2}
\rightarrow
\cdots
\rightarrow
\text{Sortie}
]

```

Le réseau est plus complexe, mais le principe d'apprentissage reste basé sur les mêmes idées :

1. effectuer une prédiction ;
2. comparer la prédiction à la vraie valeur ;
3. calculer une fonction de perte ;
4. déterminer comment modifier les paramètres ;
5. mettre à jour les paramètres ;
6. recommencer.

Les notions étudiées aujourd'hui seront donc réutilisées dans les prochains chapitres :

```

[
\boxed{
\text{Poids}
\rightarrow
\text{Biais}
\rightarrow
\text{Prédiction}
\rightarrow
\text{Loss}
\rightarrow
\text{Gradient}
\rightarrow
\text{Mise à jour}
}
]

```

Nous commençons volontairement par un problème très simple afin de pouvoir observer chaque étape de l'apprentissage.

3. Le neurone artificiel

Un neurone artificiel reçoit une ou plusieurs valeurs en entrée et produit une sortie.

Avec une seule entrée (x), il calcule :

```

[
z=wx+b
]

```

où :

- (x) est l'entrée ;
- (w) est le **poids** (*weight*) ;
- (b) est le **biais** (*bias*) ;
- (z) est la combinaison linéaire.

Dans un réseau de neurones, on applique généralement une fonction d'activation :

$$\begin{bmatrix} \hat{y} = \phi(z) \end{bmatrix}$$

soit :

$$\begin{bmatrix} \hat{y} = \phi(wx+b) \end{bmatrix}$$

Dans ce TP, nous commençons sans activation non linéaire. Nous utilisons la fonction identité :

$$\begin{bmatrix} \phi(z) = z \end{bmatrix}$$

Nous obtenons donc :

$$\begin{bmatrix} \boxed{\hat{y} = wx+b} \end{bmatrix}$$

Il s'agit d'un **neurone linéaire**.

Mathématiquement, ce modèle correspond à une régression linéaire.

Remarque : le terme *perceptron* est historiquement associé à un modèle de classification utilisant une fonction de seuil. Dans ce TP, nous utilisons un neurone linéaire pour introduire progressivement les mécanismes d'apprentissage utilisés ensuite en Deep Learning.

4. Comprendre le poids (w)

Considérons :

$$\begin{bmatrix} \hat{y} = wx+b \end{bmatrix}$$

Le poids (w) détermine l'influence de l'entrée sur la sortie.

Par exemple :

$$\begin{bmatrix} w=2, \text{ et } b=1 \end{bmatrix}$$

Pour :

$$\begin{bmatrix} x=3 \end{bmatrix}$$

le modèle prédit :

$$\begin{bmatrix} \hat{y}=2 \times 3 + 1 = 7 \end{bmatrix}$$

Si nous augmentons (x) d'une unité :

$$\begin{bmatrix} x=4 \end{bmatrix}$$

nous obtenons :

$$\begin{bmatrix} \hat{y}=2 \times 4 + 1 = 9 \end{bmatrix}$$

La sortie a augmenté de 2.

C'est précisément la valeur de (w).

Géométriquement, (w) représente donc la **pente de la droite**.

5. Comprendre le biais (b)

Le biais permet de déplacer la droite verticalement.

Considérons :

$$\begin{bmatrix} \hat{y}=2x \end{bmatrix}$$

Lorsque :

$$\begin{bmatrix} x=0 \end{bmatrix}$$

nous obtenons obligatoirement :

$$\begin{bmatrix} \hat{y}=0 \end{bmatrix}$$

La droite est donc contrainte de passer par l'origine.

Ajoutons maintenant un biais :

$$\begin{bmatrix} \hat{y}=2x+3 \end{bmatrix}$$

Pour :

$$\begin{bmatrix} x=0 \end{bmatrix}$$

nous obtenons :

$$\begin{bmatrix} \hat{y}=3 \end{bmatrix}$$

Le biais permet donc au modèle d'être plus flexible.

Géométriquement :

$$\begin{bmatrix} \boxed{\begin{aligned} w &= \text{pente} \\ \quad \quad \quad \\ b &= \text{ordonnée à l'origine} \end{aligned}} \end{bmatrix}$$

6. De la prédiction à l'erreur

Supposons que la vraie valeur soit :

$$[y=5]$$

et que notre modèle prédit :

$$[\hat{y}=7]$$

L'erreur est :

$$[e=\hat{y}-y]$$

donc :

$$[e=7-5=2]$$

Pour une autre observation, nous pourrions avoir :

$$[y=5, \text{ } \hat{y}=3]$$

et donc :

$$[e=3-5=-2]$$

Si nous calculons simplement la moyenne des erreurs, les erreurs positives et négatives peuvent s'annuler.

Nous avons donc besoin d'une meilleure mesure.

7. La fonction de perte MSE

Nous utiliserons la **Mean Squared Error (MSE)** :

$$[\boxed{MSE = \frac{1}{N}}]$$

$$\sum_{i=1}^N (\hat{y}_i - y_i)^2$$

où (N) est le nombre d'observations.

Pour chaque observation :

1. on calcule l'erreur ;
2. on élève cette erreur au carré ;
3. on calcule la moyenne.

Grâce au carré :

$$[(-2)^2 = 4]$$

et :

$$[2^2 = 4]$$

Les erreurs ne peuvent donc plus s'annuler.

La MSE possède plusieurs propriétés :

- elle est toujours positive ou nulle ;
- une petite MSE indique que les prédictions sont proches des vraies valeurs ;
- une MSE égale à zéro correspond à des prédictions parfaites ;
- elle pénalise fortement les grandes erreurs.

Notre objectif devient :

$$\boxed{\text{Trouver } w \text{ et } b \text{ qui minimisent la MSE}}$$

8. Comment le modèle apprend-il ?

Au début, nous pouvons initialiser :

$$\begin{bmatrix} w=0 \end{bmatrix}$$

et :

$$\begin{bmatrix} b=0 \end{bmatrix}$$

Le modèle :

$$\begin{bmatrix} \hat{y}=wx+b \end{bmatrix}$$

devient alors :

$$\begin{bmatrix} \hat{y}=0 \end{bmatrix}$$

Les premières prédictions seront donc généralement mauvaises.

Le modèle doit progressivement modifier (w) et (b).

Pour cela, nous utilisons la **descente de gradient**.

9. Intuition de la descente de gradient

Imaginons que la fonction de perte représente une montagne ou une vallée.

Notre position dépend des valeurs de nos paramètres.

Nous cherchons le point où la perte est minimale.

Le gradient nous indique la direction dans laquelle la fonction augmente.

Pour diminuer la perte, nous allons donc dans la direction opposée.

La mise à jour du poids est :

$$\boxed{w \leftarrow w - \rho \frac{\partial L}{\partial w}}$$

```
}  
]
```

Pour le biais :

```
[  
\boxed{  
b  
\leftarrow  
b-\rho\frac{\partial L}{\partial b}  
}  
]
```

où (ρ) représente le **learning rate**.

10. Calcul des gradients

Pour notre modèle :

```
[  
\hat{y}=wx+b  
]
```

avec la MSE, le gradient du poids est :

```
\frac{2}{N}  
\sum_{i=1}^N  
(\hat{y}_i-y_i)x_i  
]
```

et le gradient du biais :

```
\frac{2}{N}  
\sum_{i=1}^N  
(\hat{y}_i-y_i)  
]
```

Ces gradients indiquent comment modifier (w) et (b) pour réduire la perte.

11. Une itération d'apprentissage

Une itération peut être décomposée en cinq étapes.

Étape 1 : Prédiction

$$\begin{bmatrix} \hat{y} = wx + b \end{bmatrix}$$

Étape 2 : Erreur

$$\begin{bmatrix} e = \hat{y} - y \end{bmatrix}$$

Étape 3 : Loss

$$\begin{bmatrix} L = \frac{1}{N} \sum e_i^2 \end{bmatrix}$$

Étape 4 : Gradients

$$\begin{bmatrix} \frac{\partial L}{\partial w} \\ \text{et} \\ \frac{\partial L}{\partial b} \end{bmatrix}$$

Étape 5 : Mise à jour

$$\begin{bmatrix} w \leftarrow w - \rho \frac{\partial L}{\partial w} \end{bmatrix}$$

$$\begin{bmatrix} b \leftarrow b - \rho \frac{\partial L}{\partial b} \end{bmatrix}$$

Puis nous recommençons.

Une itération complète sur l'ensemble des données d'entraînement est appelée une **epoch**.

12. Le learning rate

Le *learning rate* contrôle la taille des modifications des paramètres.

Il est généralement noté :

$$\begin{bmatrix} \rho \end{bmatrix}$$

ou parfois :

[
\\eta
]

Learning rate trop petit

Les modifications de (w) et (b) sont très faibles.

Le modèle peut apprendre correctement, mais lentement.

Learning rate adapté

La perte diminue rapidement puis se stabilise.

Learning rate trop grand

Les modifications sont trop importantes.

Le modèle peut dépasser le minimum au lieu de s'en rapprocher.

La perte peut alors :

- osciller ;
- augmenter ;
- devenir instable ;
- diverger.

Le choix du learning rate est donc important.

13. Pourquoi standardiser les données ?

Les variables d'un dataset peuvent avoir des échelles très différentes.

Par exemple :

- âge : 18 à 80 ;
- surface : 20 à 300 ;
- revenu : 1 000 à 100 000.

Ces différences d'échelle peuvent compliquer l'optimisation.

Nous pouvons transformer une variable avec :

[
\\boxed{

$$x' = \frac{x - \mu}{\sigma}$$

où :

- (μ) est la moyenne ;
- (σ) est l'écart-type.

Après standardisation, nous obtenons approximativement :

$$\text{moyenne} = 0$$

et :

$$\text{écart-type} = 1$$

La standardisation facilite généralement l'optimisation, notamment lorsqu'un modèle possède plusieurs variables d'entrée.

14. Entraînement et validation

Un modèle ne doit pas uniquement être évalué sur les données qu'il a utilisées pour apprendre.

Nous séparons donc les données.

Ensemble d'entraînement (*training set*)

Il est utilisé pour apprendre :

$$w, b$$

Ensemble de validation (*validation set*)

Il permet d'évaluer le modèle sur des observations qui n'ont pas servi directement à ajuster ses paramètres.

Par exemple :

[
80% \text{ entraînement}
]

et :

[
20% \text{ validation}
]

L'objectif est de vérifier si le modèle est capable de **généraliser** à de nouvelles données.

15. MSE et MAE

Nous utiliserons deux métriques.

Mean Squared Error : MSE

[
MSE=
$$\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

]

Les grandes erreurs sont fortement pénalisées.

Mean Absolute Error : MAE

[
MAE=
$$\frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

]

La MAE utilise directement la valeur absolue de l'erreur.

Elle est généralement plus facile à interpréter puisqu'elle conserve la même unité que la variable cible.

16. Le lien avec les prochains TPs

Dans ce TP, nous avons :

$$\begin{aligned} &[\\ &x \\ &\rightarrow wx+b \\ &\rightarrow \hat{y} \\ &] \end{aligned}$$

Plus tard, nous utiliserons plusieurs entrées :

$$\begin{aligned} &[\\ &x_1, x_2, \dots, x_n \\ &] \end{aligned}$$

et donc :

$$\begin{aligned} &[\\ &z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b \\ &] \end{aligned}$$

ou, sous forme vectorielle :

$$\begin{aligned} &[\\ &z = \mathbf{w}^T \mathbf{x} + b \\ &] \end{aligned}$$

Nous ajouterons ensuite une fonction d'activation :

$$\begin{aligned} &[\\ &a = \phi(z) \\ &] \end{aligned}$$

Puis plusieurs neurones :

$$\begin{aligned} &[\\ &\boxed{ \\ &\text{Entrées} \\ &\rightarrow \\ &\text{Couche cachée} \\ &\rightarrow \\ &\text{Couche cachée} \\ &\rightarrow \\ &\text{Sortie} \\ & \\ &] \end{aligned}$$

Les calculs deviennent plus complexes, mais l'idée centrale reste :

```
[
\boxed{
\text{Prédire}
\rightarrow
\text{Mesurer l'erreur}
\rightarrow
\text{Calculer les gradients}
\rightarrow
\text{Modifier les poids}
}
]
```

C'est cette logique que nous allons maintenant observer expérimentalement.

TP GUIDÉ : Mise en pratique

1. Installation et imports

Si nécessaire :

```
# !pip install numpy pandas matplotlib scikit-learn
```

Importez ensuite les bibliothèques :

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

2. Chargement du dataset

Nous utiliserons le dataset **California Housing**.

Nous ne conservons volontairement qu'une seule variable afin de pouvoir visualiser facilement le comportement du modèle :

- AveRooms : nombre moyen de pièces ;
- MedHouseVal : valeur médiane des logements.

```
USE_SYNTHETIC = False
df = None
```

```
try:
    from sklearn.datasets import fetch_california_housing

    cal = fetch_california_housing(as_frame=True)

    df = cal.frame[['AveRooms', 'MedHouseVal']].copy()
```

```

df.rename(columns={'MedHouseVal': 'PRICE'}, inplace=True)

except Exception:

    print("Dataset indisponible : génération d'un dataset synthétique.")

    USE_SYNTHETIC = True

if USE_SYNTHETIC:

    rng = np.random.default_rng(42)

    n = 1200

    AveRooms = rng.normal(5.5, 1.2, size=n)

    PRICE = (
        0.8 * AveRooms
        + rng.normal(0.0, 0.5, size=n)
        + 1.5
    )

    df = pd.DataFrame({
        'AveRooms': AveRooms,
        'PRICE': PRICE
    })

df.sample(5)

```

À faire

1. Affichez les cinq premières lignes du dataset.
2. Affichez le nombre de lignes et de colonnes.
3. Identifiez la variable d'entrée et la variable cible.
4. Expliquez ce que représente une ligne du dataset.

3. Exploration des données

Exécutez :

```

plt.scatter(
    df['AveRooms'],
    df['PRICE'],
    alpha=0.4
)

plt.xlabel("AveRooms")
plt.ylabel("PRICE")
plt.title("Relation AveRooms → PRICE")

plt.show()

print("Taille du dataset :", df.shape)
print(df.describe())

```

À observer et commenter

1. Existe-t-il une relation visible entre `AveRooms` et `PRICE` ?
2. Cette relation semble-t-elle parfaitement linéaire ?
3. La dispersion est-elle importante ?
4. Observez-vous des valeurs atypiques ?
5. Pensez-vous qu'une seule variable puisse expliquer complètement la valeur d'un logement ? Justifiez.

4. Préparation des données

Transformez les colonnes en tableaux NumPy :

```
X = df['AveRooms'].to_numpy().astype(float)
y = df['PRICE'].to_numpy().astype(float)
```

Standardisez ensuite (X) :

```
Xn = (X - X.mean()) / X.std()
```

Vérifiez :

```
print("Avant standardisation")
print("Moyenne :", X.mean())
print("Écart-type :", X.std())

print()

print("Après standardisation")
print("Moyenne :", Xn.mean())
print("Écart-type :", Xn.std())
```

À faire

Notez la moyenne et l'écart-type avant et après standardisation.

Expliquez le résultat obtenu.

5. Premier modèle : sans biais

Nous commençons avec :

```
[
\hat{y}=wx
]
```

Implémentez la fonction suivante :

```
def train_linear_no_bias(X, y, lr=0.05, epochs=200):  
    w = 0.0  
    losses = []  
  
    for epoch in range(epochs):  
        # 1. Calculer les prédictions  
        yhat = ...  
  
        # 2. Calculer l'erreur  
        error = ...  
  
        # 3. Calculer la MSE  
        loss = ...  
  
        # 4. Calculer le gradient de w  
        grad_w = ...  
  
        # 5. Mettre à jour w  
        w = ...  
  
        losses.append(loss)  
  
    return w, np.array(losses)
```

Indications

Vous devez traduire les équations suivantes en Python :

```
[  
    \hat{y}=wx  
]
```

```
[  
    L=\frac{1}{N}\sum(\hat{y}-y)^2  
]
```

```
\frac{2}{N}  
\sum(\hat{y}-y)x  
]
```

```
[  
    w\leftarrow w-\rho\frac{\partial L}{\partial w}  
]
```

6. Observer l'apprentissage

Entraînez votre modèle puis tracez la loss :

```

w1, losses1 = train_linear_no_bias(
    Xn,
    Y,
    lr=0.05,
    epochs=200
)

print("Poids appris :", w1)

plt.plot(losses1)
plt.xlabel("Epoch")
plt.ylabel("MSE")
plt.title("Évolution de la perte")
plt.show()

```

Questions

1. La loss diminue-t-elle ?
2. Que signifie cette diminution ?
3. La diminution est-elle constante ?
4. Que se passe-t-il lorsque la courbe devient presque horizontale ?

7. Ajouter le biais

Nous passons maintenant au modèle :

```

[
 $\hat{y} = wx + b$ 
]

```

Complétez :

```

def train_linear_with_bias(X, y, lr=0.05, epochs=300):

    w = 0.0
    b = 0.0

    losses = []

    for epoch in range(epochs):

        # Prédiction
        yhat = ...

        # Erreur
        error = ...

        # Loss
        loss = ...

        # Gradients
        grad_w = ...
        grad_b = ...

```

```

        # Mise à jour
        w = ...
        b = ...

    losses.append(loss)

return w, b, np.array(losses)

```

Utilisez :

```

2, \text{mean}(\hat{y}-y)
]

```

8. Visualiser la droite apprise

Après l'entraînement :

```

w2, b2, losses2 = train_linear_with_bias(
    Xn,
    Y,
    lr=0.05,
    epochs=300
)

print("w =", w2)
print("b =", b2)

```

Créez ensuite :

```

x_plot = np.linspace(Xn.min(), Xn.max(), 200)

y_plot = w2 * x_plot + b2

plt.scatter(
    Xn,
    Y,
    alpha=0.3,
    label="Données"
)

plt.plot(
    x_plot,
    y_plot,
    linewidth=2,
    label="Modèle"
)

plt.xlabel("AveRooms standardisé")
plt.ylabel("PRICE")
plt.legend()
plt.show()

```

Questions

1. La droite représente-t-elle correctement la tendance générale ?
2. Tous les points sont-ils situés sur la droite ?
3. Pourquoi ?
4. Que représente la distance verticale entre un point et la droite ?

9. Train / validation

Utilisez :

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error
```

Séparez les données :

```
Xtr, Xva, ytr, yva = train_test_split(
    Xn.reshape(-1, 1),
    y,
    test_size=0.2,
    random_state=42
)
```

Entraînez uniquement sur `Xtr` et `ytr`.

Calculez ensuite les prédictions sur `Xva`.

À compléter

```
w3, b3, _ = train_linear_with_bias(
    Xtr.ravel(),
    ytr,
    lr=0.05,
    epochs=300
)

y_pred_val = ...

mse = mean_squared_error(yva, y_pred_val)
mae = mean_absolute_error(yva, y_pred_val)

print("MSE =", mse)
print("MAE =", mae)
```

Question

Pourquoi ne doit-on pas apprendre (w) et (b) directement sur les données de validation ?

10. Expérience 1 : Influence du learning rate

Testez :

```
learning_rates = [  
    0.0005,  
    0.005,  
    0.05,  
    0.2,  
    0.5  
]
```

Pour chaque valeur :

1. entraînez le modèle ;
2. récupérez `losses` ;
3. tracez la courbe ;
4. observez son comportement.

Vous pouvez utiliser :

```
for lr in learning_rates:  
  
    _, _, losses = train_linear_with_bias(  
        Xn,  
        Y,  
        lr=lr,  
        epochs=250  
    )  
  
    plt.plot(losses)  
    plt.title(f"Learning rate = {lr}")  
    plt.xlabel("Epoch")  
    plt.ylabel("MSE")  
    plt.show()
```

Tableau à compléter

Learning rate Convergence rapide ? Stable ? Observation

0.0005

0.005

0.05

0.2

0.5

Interprétation

Expliquez avec vos propres mots ce qui se produit lorsque le learning rate est :

- très faible ;
- adapté ;
- trop élevé.

11. Expérience 2 : Avec et sans biais

Entraînez :

Modèle A

```
[
\hat{y}=wx
]
```

Modèle B

```
[
\hat{y}=wx+b
]
```

Calculez la MSE et la MAE des deux modèles sur l'ensemble de validation.

Complétez :

Modèle MSE validation MAE validation

Sans biais

Avec biais

Questions

1. Quel effet observez-vous lorsque le biais est ajouté ?
2. Pourquoi le modèle sans biais est-il plus contraint géométriquement ?
3. Dans quel cas un modèle sans biais pourrait-il néanmoins être approprié ?

12. Expérience 3 : Effet de la standardisation

Entraînez le modèle sans standardisation :

```
w_ns, b_ns, losses_ns = train_linear_with_bias(
    X,
    Y,
    lr=0.0001,
```

```
        epochs=400
    )
```

Puis avec standardisation :

```
w_s, b_s, losses_s = train_linear_with_bias(
    Xn,
    Y,
    lr=0.05,
    epochs=400
)
```

Tracez :

```
plt.plot(
    losses_ns,
    label="Sans standardisation"
)

plt.plot(
    losses_s,
    label="Avec standardisation"
)

plt.xlabel("Epoch")
plt.ylabel("MSE")
plt.title("Effet de la standardisation")
plt.legend()

plt.show()
```

Attention

Les deux expériences utilisent ici des learning rates différents.

La comparaison permet surtout d'observer que **l'échelle des données influence le comportement de la descente de gradient et les valeurs de learning rate utilisables.**

Questions

1. Comparez les deux courbes.
2. Pourquoi avons-nous utilisé des learning rates différents ?
3. Quel est l'intérêt de la standardisation pour l'optimisation ?

13. Expérience 4 : Solution analytique

Pour une régression linéaire simple, nous pouvons également calculer directement les paramètres optimaux.

Le poids optimal est :

$$w^* = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}$$

et :

$$b^* = \bar{y} - w^* \bar{x}$$

Complétez :

```
x_mean = Xn.mean()
y_mean = y.mean()

w_star = (
    ((Xn - x_mean) * (y - y_mean)).sum()
    / ((Xn - x_mean) ** 2).sum()
)

b_star = y_mean - w_star * x_mean

print("w* =", w_star)
print("b* =", b_star)

print("w obtenu par GD =", w2)
print("b obtenu par GD =", b2)
```

Complétez le tableau :

Méthode	(w)	(b)
Descente de gradient		
Solution analytique		

Questions

1. Les valeurs sont-elles proches ?
2. Pourquoi ?
3. Si elles sont très différentes, quelles pourraient être les causes ?
4. Pourquoi utilisons-nous la descente de gradient alors qu'une solution analytique existe ici ?

14. Travail à rendre

Votre compte rendu doit contenir les éléments suivants.

Partie 1 : Exploration

Une figure représentant :

[
AveRooms $\rightarrow$ PRICE
]

accompagnée de quelques lignes d'interprétation.

Partie 2 : Learning rate

Présentez les courbes obtenues pour **au moins trois learning rates différents**.

Expliquez les différences observées.

Partie 3 : Avec et sans biais

Présentez :

Modèle **MSE** **MAE**

Sans biais

Avec biais

Interprétez les résultats.

Partie 4 : Standardisation

Présentez une figure comparant la convergence :

- avec standardisation ;
- sans standardisation.

Expliquez l'effet observé.

Partie 5 : Solution analytique

Présentez :

Méthode **(w)** **(b)**

Descente de gradient

Solution analytique

Expliquez pourquoi les résultats sont proches ou, dans le cas contraire, recherchez la cause.

Partie 6 : Conclusion

En quelques lignes, répondez à la question :

Qu'avez-vous compris du processus d'apprentissage d'un neurone à travers ce TP ?

Votre réponse doit notamment faire apparaître le lien entre :

```
[  
\text{prédiction},  
\quad  
\text{erreur},  
\quad  
\text{loss},  
\quad  
\text{gradient},  
\quad  
\text{learning rate}  
]
```

et :

```
[  
\text{mise à jour des paramètres}.  
]
```

Consignes pour le TP

Le travail doit être réalisé à partir :

- du cours ;
- des explications données pendant la séance ;
- de la documentation officielle des bibliothèques utilisées ;
- de vos propres expérimentations avec le code.

L'utilisation d'outils d'intelligence artificielle générative pour produire le code, les réponses aux questions, les interprétations ou le compte rendu n'est pas autorisée pour ce TP.

L'objectif est de comprendre personnellement chaque étape du processus d'apprentissage. Vous devez donc être capable d'expliquer le code que vous rendez, les résultats obtenus et les choix effectués.